

```
from docx import Document
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.lines import Line2D
import numpy as np
import re
import os
```

```
# -----
DOCX_PATH = "Covid_critical2.docx"
OUTPUT_PNG = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                           "forest_covid_critica.png")
# -----
```

```
#
```

```
# 1. LEITURA DO DOCX
```

```
#
```

```
def read_word_table(file_path, table_index=0):
```

```
    """
```

```
    8 colunas: grupo | variável | n_geral | n_obito | OR_IC | p_OR | ORa_IC | p_ORa
```

```
    2 linhas de cabeçalho — dados a partir da linha 2.
```

```
    Retorna também os n total e n óbitos lidos dinamicamente do cabeçalho.
```

```
    """
```

```
    doc = Document(file_path)
```

```
    table = doc.tables[table_index]
```

```
    # Lê n total e n óbitos da segunda linha de cabeçalho (linha índice 1)
```

```
    header_row1 = [c.text.strip() for c in table.rows[1].cells]
```

```
    m_ov = re.search(r'n=(\d+)', header_row1[2], re.IGNORECASE)
```

```
    m_de = re.search(r'N=(\d+)', header_row1[3], re.IGNORECASE)
```

```
    n_overall = m_ov.group(1) if m_ov else "?"
```

```
    n_deaths = m_de.group(1) if m_de else "?"
```

```
    data = []
```

```
    for row in table.rows[2:]: # pula as 2 linhas de cabeçalho
```

```
        data.append([cell.text.strip() for cell in row.cells])
```

```
    df = pd.DataFrame(data, columns=["grupo", "label", "n_geral", "n_obito",
```

```
                                   "OR_IC", "p_OR", "ORa_IC", "p_ORa"])
```

```
    return df, n_overall, n_deaths
```

```
def parse_or_ci(text):
```

```
    """
```

```
    Parseia strings do tipo '0.92 (0.55-1.53)'.
```

```
    Aceita vírgula ou ponto como decimal, e – ou - como separador do IC.
```

```
    Retorna (OR, IC_inf, IC_sup) ou (None, None, None) se inválido.
```

```
    """
```

```
    text = text.replace(",", ".").strip()
```

```
    match = re.search(r"([\d.]+)\s*((?![\d.]+)\s*[\-\,]\s*(?![\d.]+\s*))", text)
```

```
    if match:
```

```
        try:
```

```
            or_ = float(match.group(1))
```

```
            lo = float(match.group(2))
```

```

        hi = float(match.group(3))
        if or_ <= 0 or lo <= 0 or hi == float("inf") or or_ < 1e-6:
            return None, None, None
        return or_, lo, hi
    except ValueError:
        return None, None, None
return None, None, None

```

```

def parse_p(text):
    """Converte string de p-valor para float. '<0,001' vira 0.0005 (sentinela)."""
    text = text.replace(",", ".").strip()
    if text.startswith("<"):
        try:
            return float(text[1:]) / 2
        except ValueError:
            return None
    try:
        return float(text)
    except ValueError:
        return None

```

— Carregar e limpar

```

df_raw, n_total, n_deaths = read_word_table(DOCX_PATH)

```

```

# Limpar labels
df_raw["label"] = (df_raw["label"]
    .str.replace("\n", " ", regex=False)
    .str.replace(r"\s*\s*\s*\s*", "", regex=True)
    .str.strip())

```

```

# Parsear OR bruto e OR ajustado
df_raw[["OR", "IC_inf", "IC_sup"]] = df_raw["OR_IC"].apply(
    lambda x: pd.Series(parse_or_ci(x)))
df_raw[["ORa", "ICa_inf", "ICa_sup"]] = df_raw["ORa_IC"].apply(
    lambda x: pd.Series(parse_or_ci(x)))

```

```

df_raw["p_OR"] = df_raw["p_OR"].apply(parse_p)
df_raw["p_ORa"] = df_raw["p_ORa"].apply(parse_p)

```

```

df_raw["is_ref"] = False
df_raw["is_header"] = False

```

```

df_raw = df_raw.reset_index(drop=True)

```

```

print("=" * 70)
print(f"Tabela lida de: {DOCX_PATH}")
print(f"Overall n = {n_total} | Deaths n = {n_deaths}")
print(df_raw[["label", "n_geral", "n_obito", "OR", "IC_inf", "IC_sup",
    "ORa", "ICa_inf", "ICa_sup", "p_OR", "p_ORa"]].to_string())
print("=" * 70)

```

#

2. FOREST PLOT

#

— Paleta

```
BG      = "#FFFFFF"
PANEL   = "#F6F8FA"
BORDER  = "#D0D7DE"
TEXT    = "#1F2328"
SUBTEXT = "#57606A"
GOLD    = "#B08800"
COR_PROT = "#1D9E75" # OR < 1 → protetor
COR_RISCO = "#E07B39" # OR > 1 → risco
```

```
def sig_stars(p):
    if p is None: return ""
    if p < 0.001: return "****"
    if p < 0.01:  return "***"
    if p < 0.05:  return "**"
    return ""
```

— Figura: 4 colunas — [labels] | [OR bruto] | [gap] | [OR ajustado] —————

```
n_rows = len(df_raw)
fig_h = max(6, n_rows * 0.9 + 3.0)

fig, axes = plt.subplots(
    1, 4, figsize=(16, fig_h), facecolor=BG,
    gridspec_kw={"width_ratios": [2.5, 5, 2.5, 5], "wspace": 0.04}
)
ax_labels, ax_or, ax_gap, ax_ora = axes
```

```
# Coluna de espaçamento invisível
ax_gap.set_visible(False)
```

— Painel de labels (sem eixos)

```
ax_labels.set_facecolor(BG)
ax_labels.set_xlim(0, 1)
ax_labels.set_ylim(n_rows - 0.5, -0.5)
for spine in ax_labels.spines.values():
    spine.set_visible(False)
ax_labels.set_xticks([])
ax_labels.set_yticks([])

# Faixas zebreadas (sincronizadas nos 3 painéis)
for ax in (ax_labels, ax_or, ax_ora):
    for i in range(n_rows):
        bg_col = "#E8EDF2" if i % 2 == 0 else PANEL
        ax.axhspan(i - 0.42, i + 0.42, color=bg_col, alpha=0.55, zorder=0)
```

```
# Labels no painel esquerdo
for i, row in df_raw.iterrows():
    ax_labels.text(0.98, i, row["label"], color=TEXT,
                  fontsize=16, va="center", ha="right")
```

— Função genérica de painel OR

```
def draw_panel(ax, col_or, col_lo, col_hi, col_p, title, xlim):
    ax.set_facecolor(PANEL)
```

```

ax.set_xscale("log")
ax.xaxis.grid(True, color=BORDER, linewidth=0.9, zorder=0, alpha=0.8)
ax.set_axisbelow(True)
for spine in ax.spines.values():
    spine.set_edgecolor(BORDER)
    spine.set_linewidth(0.8)

ax.axvline(1.0, color=GOLD, linewidth=2.5, linestyle="--", zorder=2, alpha=0.9)

for i, row in df_raw.iterrows():
    OR = row[col_or]
    lo = row[col_lo]
    hi = row[col_hi]
    p = row[col_p]

    # OR inestimável
    if pd.isna(OR) or pd.isna(lo) or pd.isna(hi):
        ax.text(0.5, i, "_", color=SUBTEXT, fontsize=13,
                va="center", ha="center",
                transform=ax.get_yaxis_transform())
        continue

    cor = COR_PROT if OR < 1 else COR_RISCO
    sig = (p is not None) and not np.isnan(p) and (p < 0.05)

    # Barra IC capada nos limites do eixo
    lo_plot = max(lo, xlim[0] * 1.02)
    hi_plot = min(hi, xlim[1] * 0.98)
    ax.plot([lo_plot, hi_plot], [i, i],
            color=cor, linewidth=3.8, zorder=3, alpha=0.85,
            solid_capstyle="round")

    # Marcador: diamante se significativo, círculo se não
    ax.plot(OR, i,
            marker="D" if sig else "o",
            markersize=9 if sig else 7,
            color=cor,
            markerfacecolor=cor if sig else BG,
            markeredgecolor=cor,
            markeredgewidth=1.6,
            zorder=5)

    # Texto OR (IC) + estrelas à direita do painel
    hi_txt = min(hi, 9999)
    txt = f"OR:.2f ( {lo:.2f}–{hi_txt:.2f} ) {sig_stars(p)}"
    ax.text(1.02, i, txt,
            transform=ax.get_yaxis_transform(),
            color=TEXT, fontsize=15, va="center", ha="left",
            clip_on=False)

ax.set_yticks(range(n_rows))
ax.set_yticklabels([""] * n_rows)
ax.tick_params(axis="y", length=0)
ax.tick_params(axis="x", colors=SUBTEXT, labels=0)
ax.set_ylim(n_rows - 0.5, -0.5)
ax.set_xlim(*xlim)
ax.set_xlabel("Odds Ratio (scale log)", fontsize=20,
              color=SUBTEXT, labelpad=6)
ax.set_title(title, fontsize=20, fontweight="bold", color=TEXT, pad=5)

```

```
draw_panel(ax_or, "OR", "IC_inf", "IC_sup", "p_OR",
           "Crude OR (95% CI)", xlim=(0.1, 8))
draw_panel(ax_ora, "ORa", "ICa_inf", "ICa_sup", "p_ORa",
           "Adjusted OR (95% CI)", xlim=(0.1, 8))
```

— Legenda

```
legend_elements = [
    mpatches.Patch(facecolor=COR_PROT, edgecolor=COR_PROT,
                    label="Protective factor (OR < 1)"),
    mpatches.Patch(facecolor=COR_RISCO, edgecolor=COR_RISCO,
                    label="Risk factor (OR > 1)"),
    Line2D([0], [0], marker="D", color="none",
            markerfacecolor=TEXT, markeredgecolor=TEXT,
            markersize=5, label="Diamond = p < 0.05"),
    Line2D([0], [0], marker="o", color="none",
            markerfacecolor=BG, markeredgecolor=TEXT,
            markeredgewidth=1.2, markersize=6,
            label="Open circle = p ≥ 0.05"),
    Line2D([0], [0], color=GOLD, linewidth=1.2,
            linestyle="--", label="Reference line (OR = 1)"),
]
ax_or.legend(handles=legend_elements, fontsize=8, frameon=True,
             edgecolor=BORDER, facecolor=BG, labelcolor=TEXT,
             loc="lower left", framealpha=0.97,
             borderpad=0.9, handlelength=0.5)
```

— Título geral

```
fig.text(0.50, 1.095,
        "Forest Plot — Crude and Adjusted Odds Ratios",
        ha="center", va="top",
        fontsize=30, fontweight="bold", color=TEXT)
```

— Subtítulo dinâmico (n lido do cabeçalho da tabela)

```
subtitle = (f"Critical COVID-19 (binary variables) | "
            f"Overall n = {n_total} | Deaths n = {n_deaths}")
fig.text(0.50, 1.015, subtitle,
        ha="center", va="top", fontsize=20, color=SUBTEXT)
```

```
fig.add_artist(plt.Line2D(
    [0.13, 0.97], [0.958, 0.958],
    transform=fig.transFigure, color=BORDER, linewidth=1.8))
```

```
fig.text(0.03, -0.101,
        "*** p<0.001 ** p<0.01 * p<0.05 | "
        "OR = Odds Ratio; CI = 95% Confidence Interval | "
        "Filled diamond = p < 0.05 | — = Unestimable OR",
        color=SUBTEXT, fontsize=12, style="italic")
```

```
plt.tight_layout(rect=[0, 0.018, 1, 1.970])
plt.savefig(OUTPUT_PNG, dpi=180, bbox_inches="tight", facecolor=BG)
print(f"\n Gráfico salvo em: {OUTPUT_PNG}")
plt.show()
```